Stock Project

Team name: Stock Project | **Team member:** Scottie Dunnavan | **Deliverable:** Project Update & Preliminary Solution (`report_ps4.pdf`)

PDF

1. Problem Statement

Retail traders compete against institutions that run algorithmic and high-frequency strategies on proprietary data and infrastructure. Manual chart reading cannot keep pace with the volume of live order-book updates and is prone to emotional bias. This project targets one narrow, measurable version of that gap: short-horizon (1–5 minute intraday, standardized at a shared **3-minute / 180-second** evaluation horizon) price-direction prediction on small-to-mid-cap U.S. equities ($1M$–500M market cap)[cite: 20, 22, 23]. Given recent limit order book (LOB) and price-history data for a stock, competing models estimate the probability that the price will achieve a spread-clearing move Up, remain Neutral, or move Down, surfacing alerts under strict risk limits[cite: 20, 22].

Why it matters: Whether deep learning can extract a usable signal from data an individual can actually access, and how much of the published LOB results survive outside curated benchmarks, are open and practical questions. Because financial data is noisy and non-stationary, the project emphasizes rigorous, leakage-free evaluation over headline accuracy; a well-evaluated negative or marginal result is still a valid outcome.

2. Motivation & Objectives

Motivation: I have a long-standing interest in the mathematics of markets, inspired by quantitative firms that trade systematically using mathematical and statistical-learning methods. This is a personal hobby and portfolio project to learn the foundations of algorithmic trading and deep learning by building an end-to-end system (advisory/paper trading only in this course). LOB data is high-dimensional (prices and volumes across many levels), sequential, and noisy. Hand-crafted indicators discard much of that structure, while convolutional and recurrent/attention models can learn spatial and temporal patterns directly. DeepLOB [1] showed this on benchmark data; this project tests how far it carries to live U.S. equity data.

Objectives (task type: 3-class classification of price direction at shared horizon $\tau = 180s$ $\tau = 180s$):[cite: 20, 22]

- **O1 (core):** Reproduce DeepLOB on the FI-2010 benchmark to validate the pipeline.
  O2 (core): Build classical baselines (Random Forest, XGBoost) on technical and order-book microstructure features[cite: 20, 22].
- **O3 (core):** Train complementary deep models on U.S. equity data predicting the shared 3-minute horizon: **DeepLOB** (60-second lookback of 100ms L2/L1 ticks, with spatial CNN layers unfrozen at 0.1×0.1× learning rate during transfer learning) and **LSTM/TFT** (30–60 minute lookback of 1-minute OHLCV+RVOL bars)[cite: 20, 22].
- **O4 (core):** Evaluate with chronological splits, Macro-F1, confusion matrices, and spread-adjusted transaction costs[cite: 9, 20, 22].
- **O5 (stretch):** Fuse micro (LOB) and macro (price-history) signals on a live monitoring dashboard for paper-trading validation.
-

3. Dataset Description

The data plan has three stages: a public benchmark (FI-2010) to validate the pipeline, a purchased historical pull (Databento) to bridge the gap while recording begins, and self-recorded live U.S. equity data for the primary experiments, supported by free 1-minute/daily historical data.

Phase 1 — FI-2010 benchmark (validation): 5 Nasdaq Nordic stocks, 10 trading days (June 2010), ~4 million order-book events reduced to ~396,000 normalized time steps; top-10 bid/ask price and volume levels; Up/Stationary/Down labels [2]. Used to reproduce DeepLOB and pretrain spatial order-book weights.
PDF+ 2

- **Phase 1.5 — Databento historical pull (bridging):** Historical Level 1 and Level 2 (Nasdaq TotalView-ITCH) small-cap data pulled via our $125 Databento training credit to end-to-end test transfer learning immediately while live recordings accumulate.
  PDF+ 2
- **Phase 2 — Self-recorded U.S. equity data (primary, ongoing):** A custom `asyncio`-based Python ingestion engine streams live market data across four sources:

Source	Contents	Cost / Constraints	Role
Moomoo OpenAPI	NYSE ArcaBook order-book depth (Level 2)	Free with minimum balance; local OpenD gateway PDF+ 1	Primary L2 depth, venue 1 of 2 (100-slot pool)[cite: 20, 23]
Interactive Brokers (IBKR)	Nasdaq TotalView order-book depth (Level 2)	Paid subscription (~ $17.50–22$/mo, 3-slot cap) PDF+ 1	Primary L2 depth, venue 2 of 2 (top-3 priority pool)[cite: 20, 23]
tastytrade Open API	Real-time consolidated Level 1 quotes & trades	Free (CTA+UTP NBBO via dxLink) PDF+ 1	Universal NBBO L1 quote/trade anchor for all tickers PDF+ 1
Yahoo / IBKR History	1-minute & Daily OHLCV history	Free; rate-limited PDF+ 1	Macro sequence model (`LSTM/TFT`) & baseline features[cite: 8, 20, 23]

Refined Schema & Data-Integrity Upgrades (Updated in PS4): To resolve the buffered file-writing and cross-feed synchronization issues identified in PS3, four upgrades were locked into the recording and preprocessing schema:

1. **Dual Clocks & Lee-Ready Aggressor Classification:** Every row records both the exchange timestamp (`exch_ts_ns`) and local monotonic clock (`recv_mono_ns`). `l1_quotes.parquet` flags quote updates vs. trade prints (`event_type`) and classifies trade direction (`trade_side` $\in\{+1,-1,0\}\in\{+1,-1,0\}$) using the Lee-Ready Quote + Tick rule against prevailing NBBO.
   Sonar Stock Project — Technical Coding Specification (v1 & v2)
2. **Flat 45-Column Wide L2 Schema (`l2_snapshots.parquet`):** Each 10-level L2 update is stored as a single flat row (`bid_px_1..10`, `bid_sz_1..10`, `ask_px_1..10`, `ask_sz_1..10`, padded with `NaN`/`0` on sparse books) and tagged by venue

(`moomoo_arca` or `ibkr_totalview`) without pre-summing across exchanges.
Sonar Stock Project — Technical Coding Specification (v1 & v2)

3. **Crash-Safe Atomic Parquet Flush:** Data flushes write to temporary `.parquet.tmp` files (`zstd` compressed) and execute an atomic OS rename (`os.replace()`), preventing corrupted or overwritten files[cite: 15, 20].
4. **600-Second Minimum L2 Dwell Lock:** Promoted Tier 2/3 tickers hold their L2 slot for a minimum of 600 seconds (60s window+300s max horizon+240s buffer60s window+300s max horizon+240s buffer) so no recording slice is cut off before its forward label completes[cite: 19, 20].

4. Preliminary Solution & Preprocessing Implementation (`src/`)

For Stage 4 (`ps4-sol1`), a runnable preprocessing, feature-engineering, and baseline classification pipeline has been added in `src/preprocessing.py` and wired into `src/main.py`:

- **Microstructure Feature Extraction:** To handle sparse small-cap order books where adjacent occupied levels have uneven price gaps, raw prices are transformed into **Relative Price Distance** Δp~i=(pi−pmid)/pmidΔp~i=(pi−pmid)/pmid and log-volumes ln(1+vi)ln(1+vi). The pipeline also computes 10-level **Multi-Level Order Flow Imbalance (`MLOFI`)**, **Micro-Price Divergence**, and the **Absorption & Iceberg Ratio (`AIR`)**, which divides aggressive L1 trade volume at price pp by visible L2 depth depletion (AIR≥1.0AIR≥1.0 detects hidden iceberg absorption; AIR≤0.05AIR≤0.05 on large wall drops flags canceled spoofs)[cite: 8, 20].
  Sonar Stock Project — Technical Coding Specification (v1 & v2)
- **Spread-Adjusted Triple-Barrier Labeling:** Anchored on consolidated NBBO, a time step tt is labeled **Up (`2`)** only if the future Best Bid clears the current Best Ask by a volatility-scaled threshold θt=max(15 bps,kσt)θt=max(15 bps,kσt) within τ=180sτ=180s before the future Best Ask breaches the lower stop barrier (**Down (`0`)**), and **Neutral (`1`)** otherwise.
  Sonar Stock Project — Technical Coding Specification (v1 & v2)
- **Runnable Execution & Single-Session API Isolation (`python src/main.py`):** Broker APIs (IBKR Gateway, Moomoo OpenD, and Webull) enforce strict single-active-session limits per account; sharing live `.env` credentials or launching concurrent external connections between 4:00 AM and 8:00 PM ET (or during overnight sessions) would forcibly disconnect the active data recorder. Therefore, running `python src/main.py` executes a **self-contained offline verification pipeline by default** (requiring zero API credentials and causing zero session conflicts): it ingests a sample small-cap L1+L2 stream, writes atomic `.parquet` partitions to `src/recordings/`, constructs both the 4D DeepLOB tensor `(Batch, 1, 100, 40)` and 2D baseline feature matrix `(Batch, 46)`, fits a preliminary chronological Random Forest baseline, and prints the resulting Macro-F1 score and 3×33×3confusion matrix. Passing `python`

`src/main.py --live` with local `.env` credentials activates the live Tastytrade streamer.

References

[1] Zhang, Z., Zohren, S., & Roberts, S. (2019). DeepLOB: Deep Convolutional Neural Networks for Limit Order Books. *IEEE Transactions on Signal Processing*, 67(11), 3001–3012.

PDF

[2] Ntakaris, A., Magris, M., Kanniainen, J., Gabbouj, M., & Iosifidis, A. (2018). Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods. *Journal of Forecasting*. Dataset: `etsin.fairdata.fi/dataset/73eb48d7-4dbc-4a10-a52a-da745b47a649`.

PDF